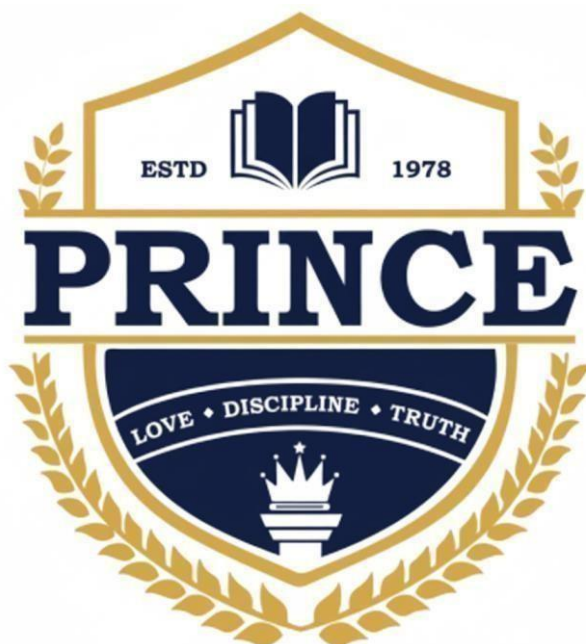


PRINCE SHRI VENKATESHWARA
ARTS AND SCIENCE COLLEGE

GOWRIVAKKAM, CHENNAI-600073.

(AFFILIATED TO THE UNIVERSITY OF MADRAS)

DEPARTMENT OF COMPUTER APPLICATIONS



WEB TECHNOLOGY PRACTICAL (320C51)

ACADEMIC YEAR

2026-2027

Name:_____

Class :_____



PRINCE SHRI VENKATESHWARA ARTS AND SCIENCE COLLEGE

BHARATHIYAR STREET, SIVAGAMI NAGAR, GOWRIVAKKAM, CHENNAI - 600 073.

Approved by the Govt. of Tamil Nadu / Affiliated to University of Madras / NAAC Accredited with A+ Grade
NIRF Band : 101-150 / ISO 9001 : 2015 Certified Institution / Recognized by UGC under section 2(f) UGC Act 1956

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

REGISTER NUMBER: _____

This is to certify that _____

Studying in **III BCA** in this college has successfully completed his / her “**WEB TECHNOLOGY PRACTICAL (320C51)**” record for the **Semester – V** during the academic year **2026 - 2027**.

Faculty Incharge

Head of the Department

Submitted for the practical examination held on _____ at

Prince Shri Venkateshwara Arts and Science College.

INTERNAL EXAMINER

EXTERNAL EXAMINER

S.NO	DATE	TOPIC	PAGE NO.	SIGN
01	15-06-2026	SUM OF ROWS AND COLUMNS IN A MATRIX	01	
02	16-06-2026	SUM OF THE FIRST N PRIME NUMBERS	04	
03	22-06-2026	PROGRAM TO CHECK WHETHER AN EMAIL ADDRESS IS VALID OR NOT	07	
04	29-06-2026	PROGRAM TO CONVERT A NUMBER WRITTEN IN WORDS TO DIGITS	10	
05	01-07-2026	PROGRAM TO DISPLAY START AND END TIME WITH A DELAY	13	
06	06-07-2026	PHP PROGRAM TO CHANGE THE COLOR OF FIRST CHARACTERS OF A WORD	16	
07	07-07-2026	PHP PROGRAM TO DISPLAY THE MULTIPLICATION TABLE OF NUMBER	20	
08	13-07-2026	PHP PROGRAM TO CALCULATE THE FACTORIAL OF A GIVEN NUMBER	23	
09	14-07-2026	SIMPLE PHP SCRIPT TO REVERSE TEXT FROM A FILE AND SAVE IT TO ANOTHER FILE	26	
10	20-07-2026	RENAME .TXT FILES TO .XTX IN CURRENT DIRECTORY	29	
11	27-07-2026	PHP PROGRAM TO LIST FILES IN THE CURRENT DIRECTORY SORTED BY LAST MODIFICATION TIME	32	
12	03-08-2026	PROGRAM TO CREATE A STUDENT MARK SHEET TABLE	35	
13	10-08-2026	PROGRAM TO RETRIEVE AND PRINT ALL THE EMAIL ADDRESSES FROM AN XML DOCUMENT USING PHP	39	
14	17-08-2026	PROGRAM TO RETRIEVE THE TEXT VALUE "JOHN" FORM AN XML DOCUMENT USING DOM IN PHP	42	
15	24-08-2026	PROGRAM TO CONNECT PHP WITH MYSQL AND SAVE TABLE DATA AS XML USING DOM	45	

Aim:

To write a PHP program to accept four numbers and display their row-wise, column-wise and total sums in a table.

Algorithm:

Step 1: Start the program.

Step 2: Create an HTML form to enter four numbers A, B, C and D.

Step 3: Submit the values using the GET method.

Step 4: Retrieve A, B, C and D using `$_GET`.

Step 5: Calculate $A + B$ and $C + D$.

Step 6: Calculate $A + C$ and $B + D$.

Step 7: Calculate the total of $A + B + C + D$.

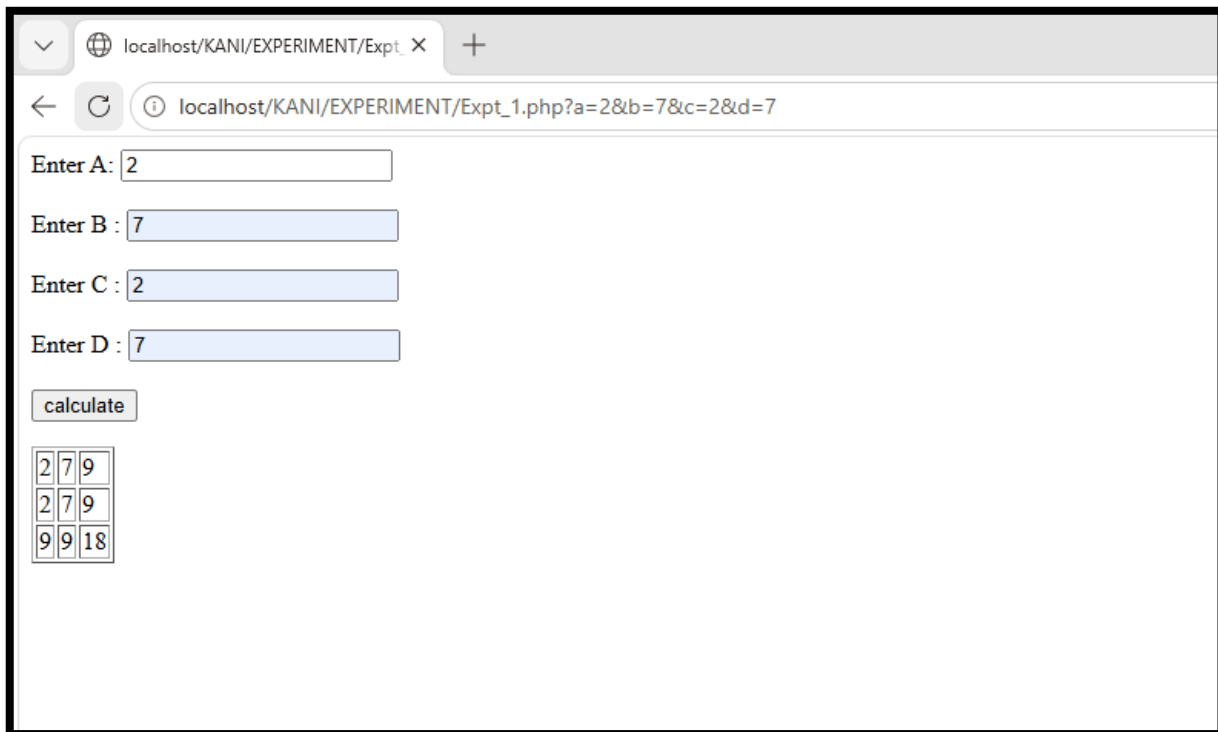
Step 8: Display the values and calculated results in an HTML table.

Step 9: Stop the program.

PROGRAM:

```
<html>
<body>
<form method="get">
Enter A:<input type="number" name="a"><br><br>
Enter B:<input type="number" name="b"><br><br>
Enter C:<input type="number" name="c"><br><br>
Enter D:<input type="number" name="d"><br><br>
<input type="submit" value="calculate">
</form>
<?php
if(isset($_GET['a']))
{
$a=$_GET['a'];
$b=$_GET['b'];
$c=$_GET['c'];
$d=$_GET['d'];
echo"<table border='1'>";
echo"<tr>";
echo"<td>$a</td><td>$b</td><td>".($a+$b)."</td>";
echo"</tr>";
echo"<tr>";
echo"<td>$c</td><td>$d</td><td>".($c+$d)."</td>";
echo"</tr>";
echo"<tr>";
echo"<td>".($a+$c)."</td><td>".($b+$d)."</td><td>".($a+$b+$c+$d)."</td>";
echo"</tr>";
echo"</table>";
}
?>
</body>
</html>
```

OUTPUT:



localhost/KANI/EXPERIMENT/Expt. x +

localhost/KANI/EXPERIMENT/Expt_1.php?a=2&b=7&c=2&d=7

Enter A:

Enter B :

Enter C :

Enter D :

2	7	9
2	7	9
9	9	18

Result:

Thus, the PHP Program has been executed successfully.

AIM:

To write a PHP program to find the **sum of the first n prime numbers**.

ALGORITHM:

Step 1: Start the program.

Step 2: Accept the value of N from the user.

Step 3: Initialize count = 0, num = 2 and sum = 0.

Step 4: Check whether count < N.

Step 5: Assume the current number is prime.

Step 6: Check whether the number is divisible by any number from 2 to num - 1.

Step 7: If divisible, mark the number as non-prime.

Step 8: If the number is prime, add it to sum and increment count.

Step 9: Increment num by 1.

Step 10: Repeat Steps 4 to 9 until N prime numbers are found.

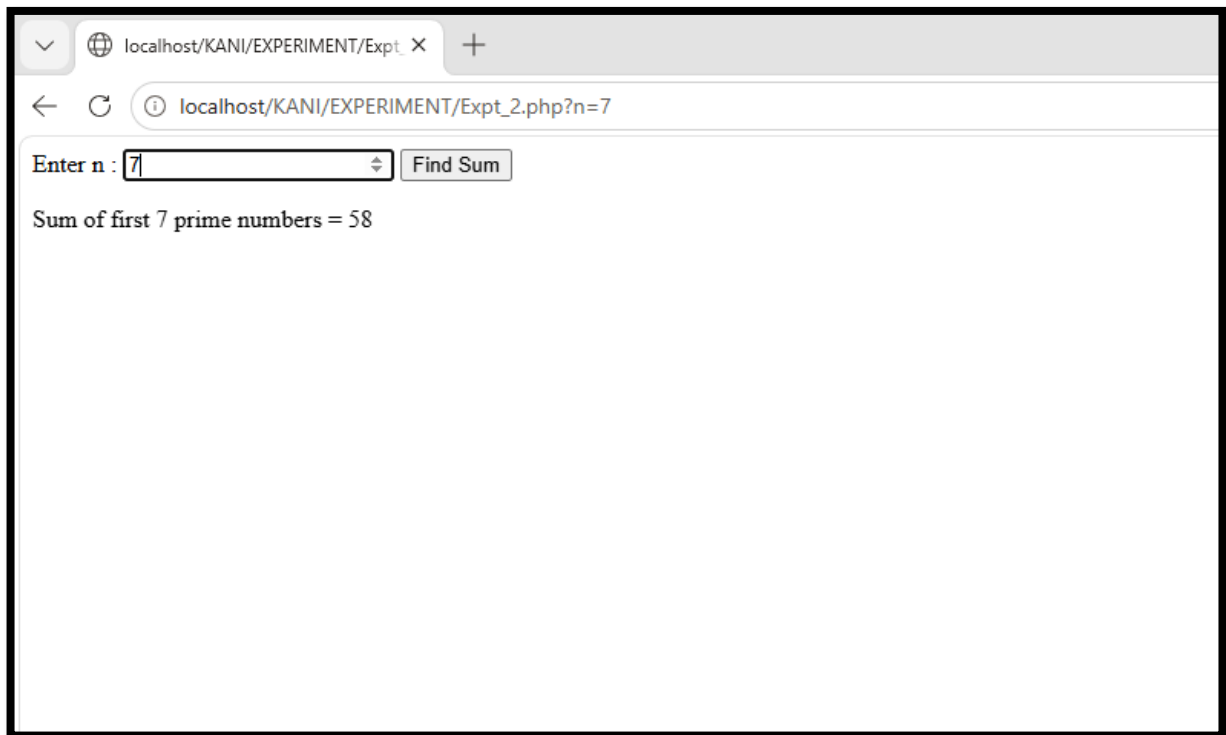
Step 11: Display the sum of the first N prime numbers.

Step 12: Stop the program

PROGRAM:

```
<html>
<body>
<form method="get">
Enter n:
<input type="number" name="n">
<input type="submit" value="Find sum">
</form>
<?php
if(isset($_GET['n'])) {
    $n=$_GET['n'];
    $count=0;
    $num=2;
    $sum=0;
    while($count<$n){
        $prime=1;
        for($i=2;$i<$num;$i++){
            if($num%$i==0){
                $prime=0;
                break;
            }
        }
        if($prime==1){
            $sum=$sum+$num;
            $count++;
        }
        $num++;
    }
    echo "sum of first $n prime numbers=$sum";
}??>
</body>
</html>
```

OUTPUT:



RESULT:

Thus, the PHP Program has been executed successfully.

Aim:

To write a PHP program to validate an email address.

Algorithm:

Step 1: Start the program.

Step 2: Create a form to enter an email address.

Step 3: Submit the email using the POST method.

Step 4: Retrieve the email using \$_POST.

Step 5: Validate the email using FILTER_VALIDATE_EMAIL.

Step 6: If the email is valid, display "**Valid Email Address**".

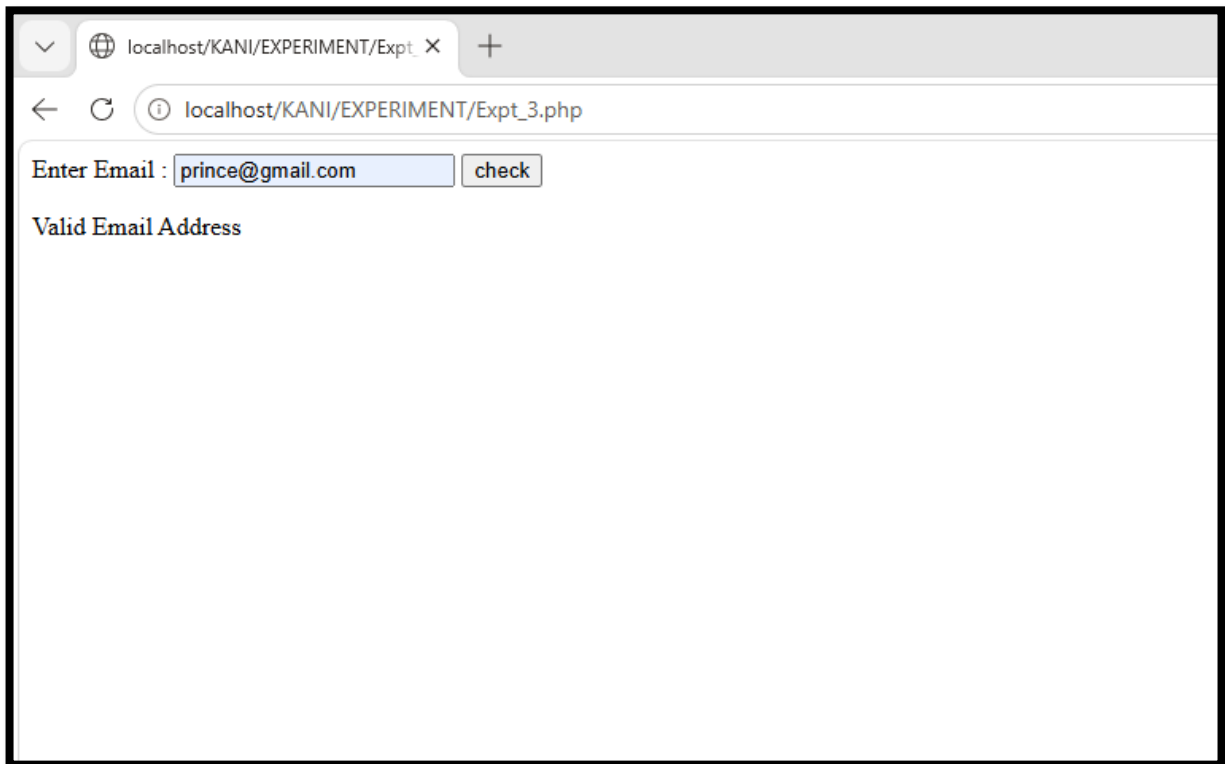
Step 7: Otherwise, display "**Invalid Email Address**".

Step 8: Stop the program.

PROGRAM:

```
<html>
<body>
<form method ="post">
Enter Email:
<input type="text" name="email">
<input type="submit" value="check">
</form>
<?php
if(isset($_POST['email'])) {
$email=$_POST['email'];
if(filter_var($email,FILTER_VALIDATE_EMAIL))
echo"valid Email Address";
else
echo"Invalid Email Address";
}
?>
</body>
</html>
```

OUTPUT:



A screenshot of a web browser window. The address bar shows 'localhost/KANI/EXPERIMENT/Expt_3.php'. The page content includes a form with the label 'Enter Email :', a text input field containing 'prince@gmail.com', and a 'check' button. Below the form, the text 'Valid Email Address' is displayed, indicating a successful validation.

Result:

Thus, the PHP Program has been executed successfully

Program To Convert a Number written in Words to Digits

Aim:

To write a PHP program to convert a number entered in words into its corresponding digit.

Algorithm:

Step 1: Start the program.

Step 2: Create a form to enter a number in words.

Step 3: Retrieve the input using `$_POST`.

Step 4: Convert the input into lowercase using `strtolower()`.

Step 5: Use a switch statement to compare the entered word.

Step 6: If the word is "zero", display 0.

Step 7: If the word is "one", display 1 and continue similarly up to "ten".

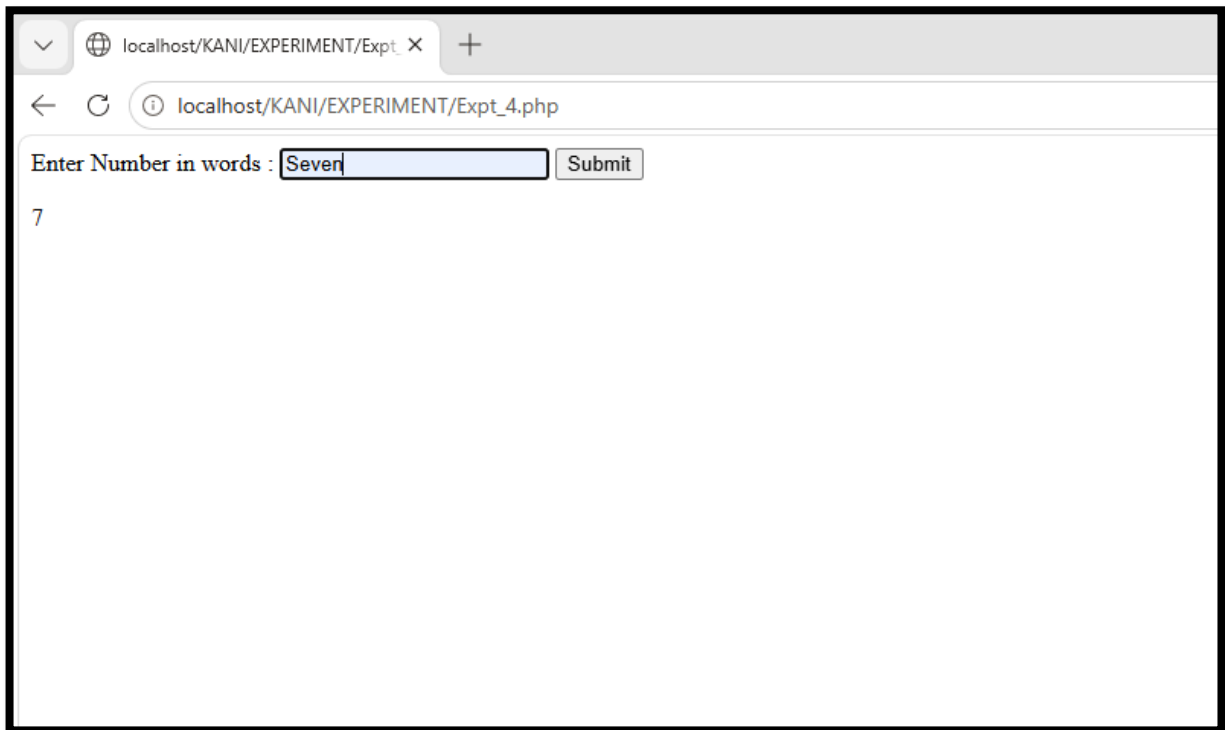
Step 8: If the word does not match any case, display **"Invalid Input"**.

Step 9: Stop the program.

PROGRAM:

```
<html>
<body>
<form method="post">
Enter numbers in words:
<input type="text" name="word">
<input type="submit" value="convert">
</form>
<?php
if(isset($_POST['word'])) {
$word=strtolower($_POST['word']);
switch($word) {
case"zero":echo"0";break;
case"one":echo"1";break;
case"two":echo"2";break;
case"three":echo"3";break;
case"four":echo"4";break;
case"five":echo"5";break;
case"six":echo"6";break;
case"seven":echo"7";break;
case"eight":echo"8";break;
case"nine":echo"9";break;
case"ten":echo"10";break;
default:echo"invalid input";
}
}
?>
</body>
</html>
```

OUTPUT:



A screenshot of a web browser window. The address bar shows 'localhost/KANI/EXPERIMENT/Expt_4.php'. The page content includes a form with the label 'Enter Number in words :', a text input field containing the word 'Seven', and a 'Submit' button. Below the form, the number '7' is displayed as the output.

Result:

Thus, the PHP Program has been executed successfully.

AIM:

To write a PHP program that shows the start time, waits for 5 seconds, and then shows the end time.

ALGORITHM:

Step-1 : Start

Step-2 : Print the current time as the start time

Step-3 : Pause the program execution for some seconds.

Step-4 : After the delay, print the current time as the end time

Step-5 : Stop

PROGRAM:

```
<html>

<body>

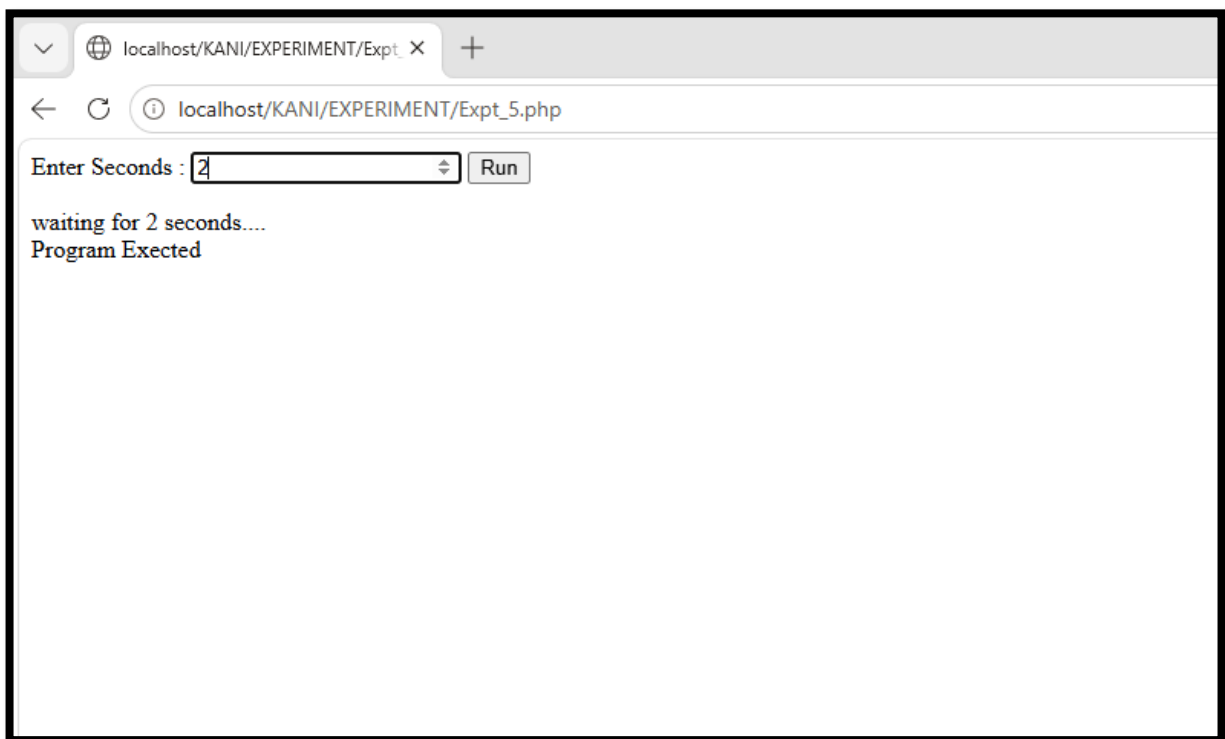
<form method="post">
Enter seconds:
<input type="number" name="sec">
<input type="submit" value="Run">
</form>

<?php
if(isset($_POST['sec']))
{
$sec=$_POST['sec'];
echo"waiting for $sec seconds...<br>";
sleep($sec);
echo"program executed";
}
?>

</body>

</html>
```

OUTPUT:



Result:

Thus, the PHP Program has been executed successfully.

AIM:

To write a PHP program to display the first character of a word in a different color.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a form to enter a word.

Step 3: Retrieve the word using `$_POST`.

Step 4: Extract the first character using `$word[0]`.

Step 5: Display the first character using a `<span>` tag with a color style.

Step 6: Extract the remaining characters using `substr()`.

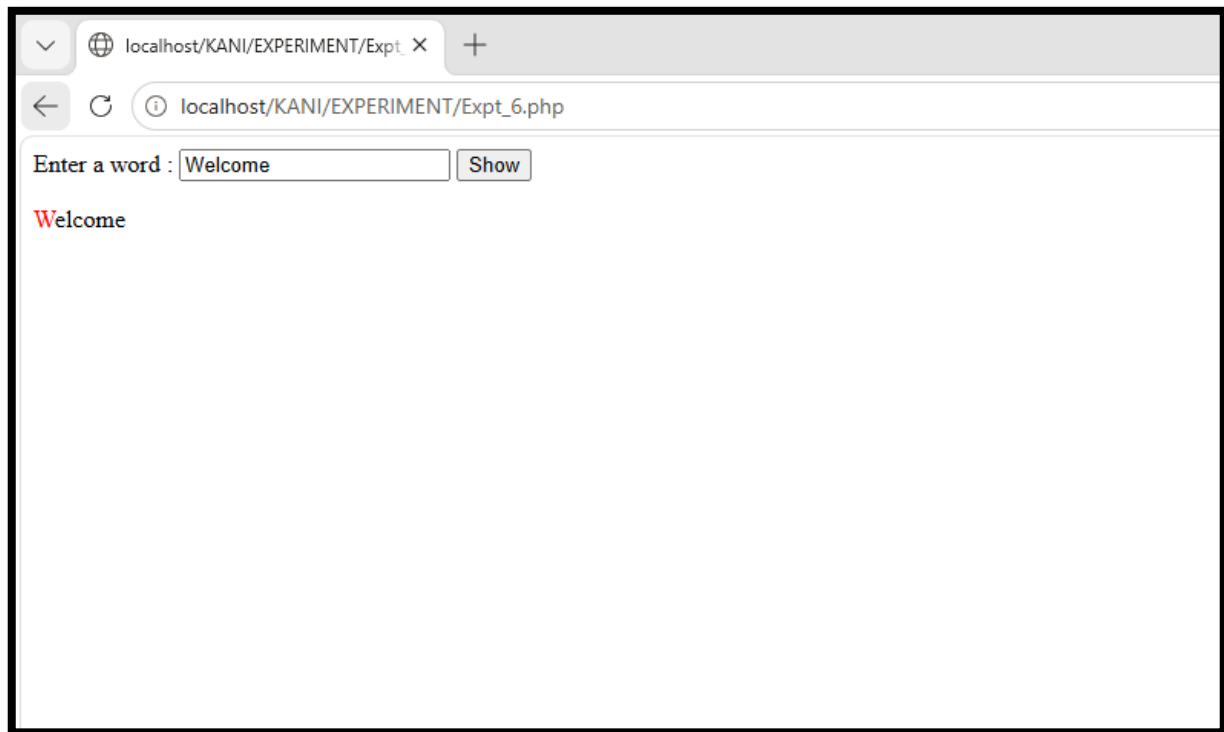
Step 7: Display the remaining characters normally.

Step 8: Stop the program.

PROGRAM:

```
<html>
<body>
<form method="post">
Enter a word:
<input type="text" name="word">
<input type="submit" value="show">
</form>
<?php
if(isset($_POST['word']))
{
$word=$_POST['word'];
echo"<span style='color:red'>".$_word[0]."</span>".substr($word,1);
}
?>
</body>
</html>
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.

AIM:

To write a php program that takes a number as input and display its multiplication table.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a form to enter a number.

Step 3: Retrieve the number using `$_POST`.

Step 4: Initialize `i = 1`.

Step 5: Check whether `i <= 10`.

Step 6: Multiply the given number by `i`.

Step 7: Display the multiplication result.

Step 8: Increment `i` by 1.

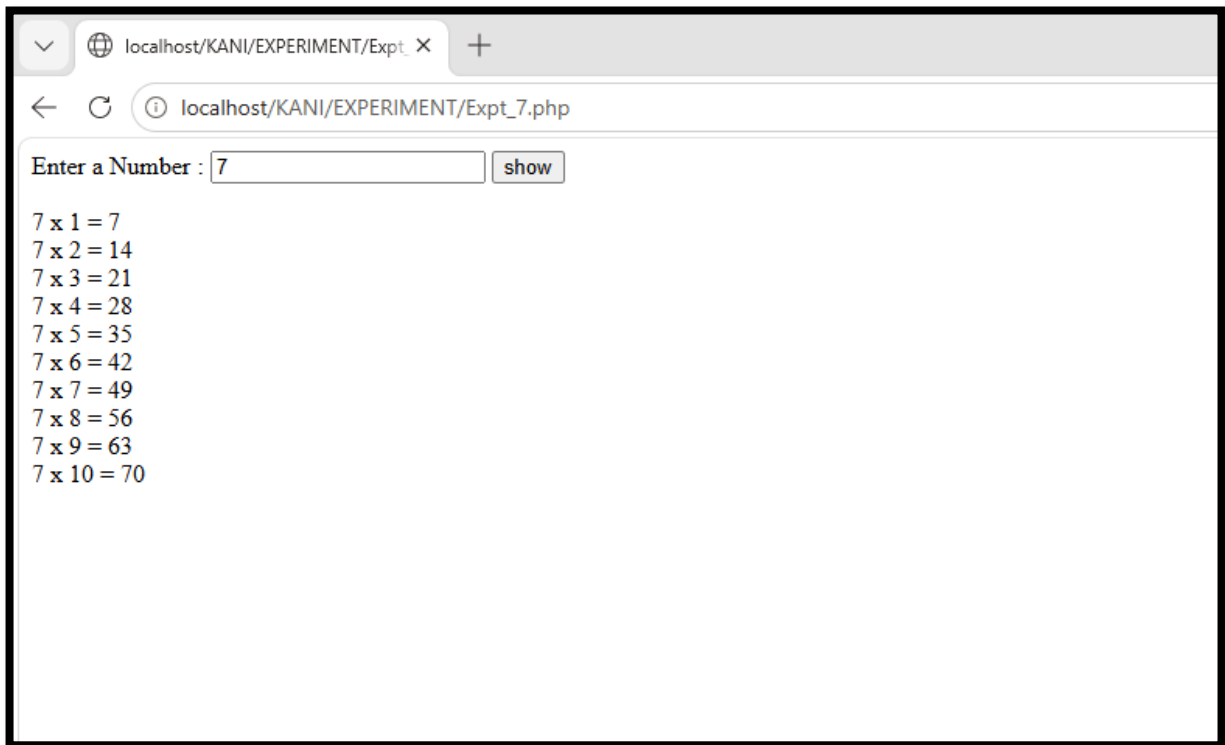
Step 9: Repeat Steps 5 to 8 until `i` becomes 11.

Step 10: Stop the program.

PROGRAM:

```
<html>
<body>
<form method="POST">
enter a number:<input type="number"name="num">
<input type="submit"value="show">
</form>
<?php
if(isset($_POST['num'])) {
$num=$_POST['num'];
for($i=1;$i<=10;$i++)
echo"$num*$i=".( $num*$i )."<br>";
}
?>
</body>
</html>
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.

AIM:

To write a PHP program to find the factorial of a given number.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a form to enter a number.

Step 3: Retrieve the number using `$_POST`.

Step 4: Initialize `fact = 1`.

Step 5: Initialize `i = 1`.

Step 6: Check whether `i <= num`.

Step 7: Multiply `fact` by `i`.

Step 8: Increment `i` by 1.

Step 9: Repeat Steps 6 to 8 until `i` becomes greater than `num`.

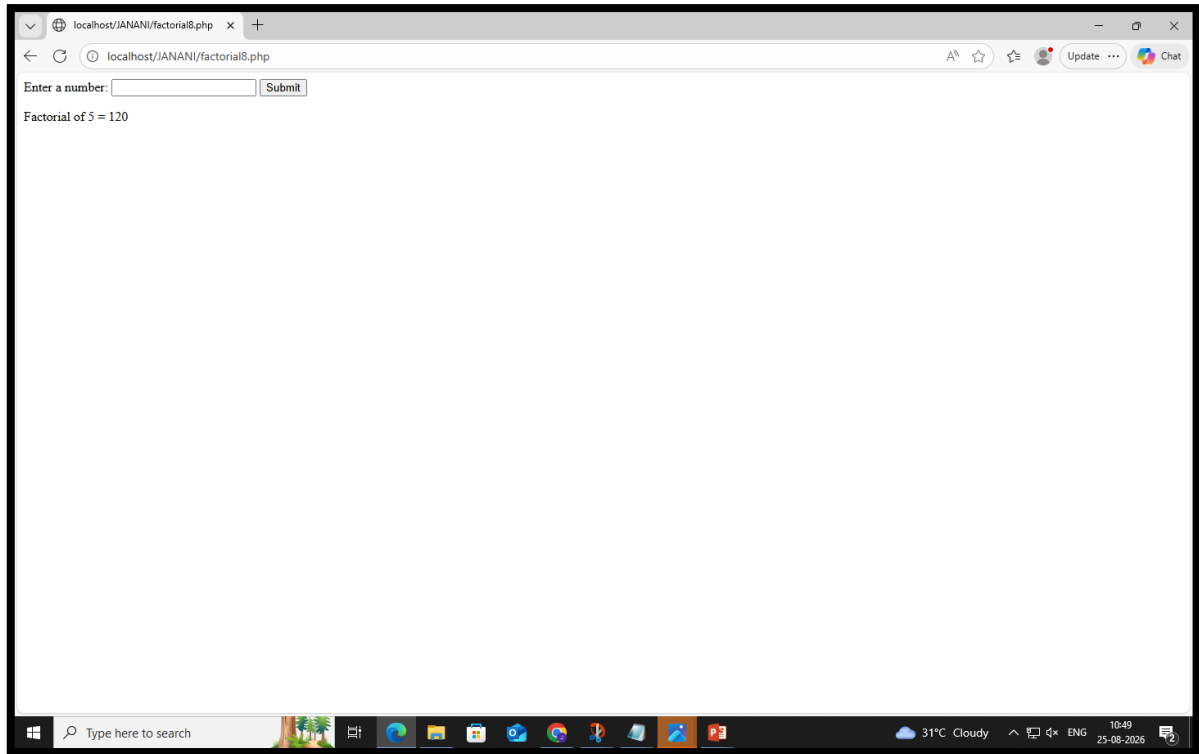
Step 10: Display the factorial of the given number.

Step 11: Stop the program.

PROGRAM:

```
<html>
<body>
<form method="post">
enter a number:
<input type="number" name="num">
<input type="submit" value="show">
</form>
<?php
if(isset($_POST['num'])) {
$num=$_POST['num'];
$fact=1;
for($i=1;$i<=$num;$i++)
$fact*=$i;
echo"factorial of $num=$fact";
}
?>
</body>
</html>
```

OUTPUT:



Result:

Thus, the PHP Program has been executed successfully.

AIM:

To write a PHP program to reverse the contents of a text file and store the reversed contents in another file..

ALGORITHM:

Step 1: Start the program.

Step 2: Specify the input file name.

Step 3: Specify the output file name.

Step 4: Read the contents of the input file using `file_get_contents()`.

Step 5: Reverse the contents using `strrev()`.

Step 6: Store the reversed contents in the output file using `file_put_contents()`.

Step 7: Display "**Contents reversed successfully**".

Step 8: Stop the program.

PROGRAM:

```
<?php
$inputFile="sample1.txt";
$outputFile="rev1.txt";
$data=File_get_contents($inputFile);
$reverse=strrev($data);
File_put_contents($outputFile,$reverse);
echo"contents reversed successfully.";
?>
```

OUTPUT:



Result:

Thus, the PHP Program has been executed successfully

Aim:

To write a PHP program to rename all .txt files to .xtx files.

Algorithm:

Step 1: Start the program.

Step 2: Find all .txt files using `glob("*.*txt")`.

Step 3: Read each file name using a foreach loop.

Step 4: Replace the .txt extension with .xtx.

Step 5: Rename the file using the `rename()` function.

Step 6: Display the old and new file names.

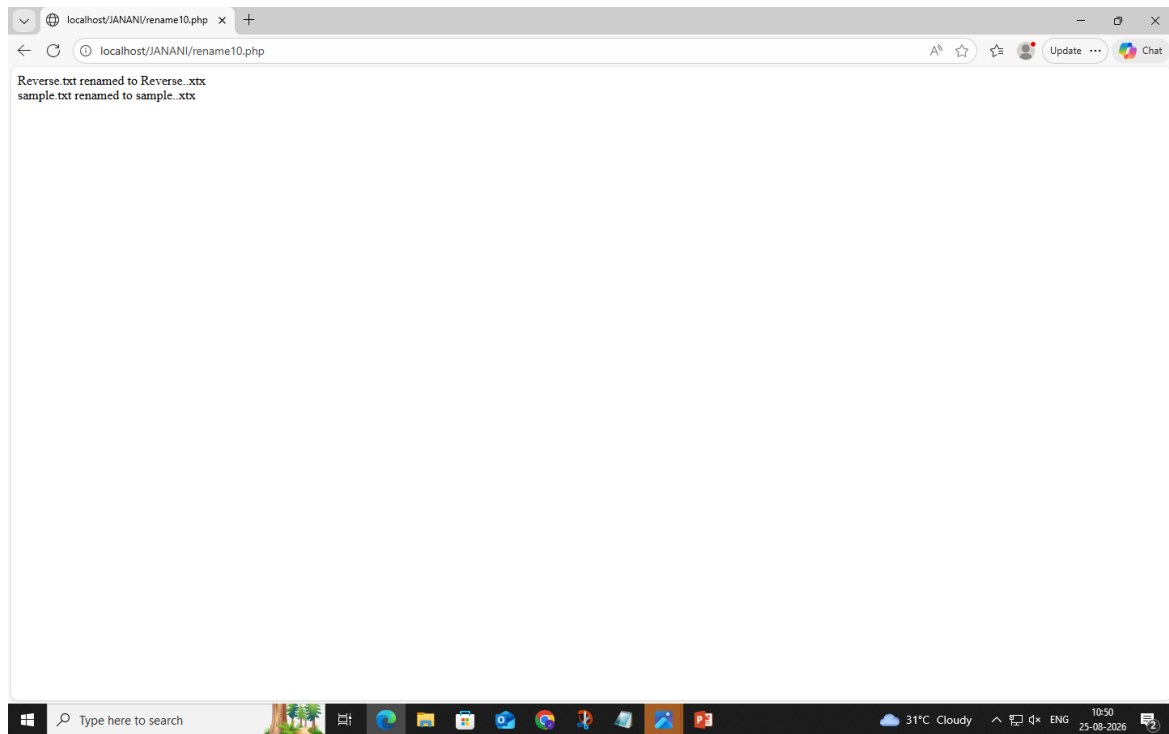
Step 7: Repeat Steps 3 to 6 for all .txt files.

Step 8: Stop the program.

PROGRAM:

```
<?php
$files=glob("*.txt");
foreach($files as $file){
$newname=str_replace(".txt",".txt",$file);
rename($file,$newname);
echo"$file renamed to $newname <br>";
}
?>
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.

AIM:

To write a PHP program to display files sorted according to their last modification time.

ALGORITHM:

Step 1: Start the program.

Step 2: Create an empty array to store file names and modification times.

Step 3: Open the current directory using opendir().

Step 4: Read the files using readdir().

Step 5: Ignore the . and .. entries.

Step 6: Find the modification time of each file using filemtime().

Step 7: Store the file name and modification time in the array.

Step 8: Sort the array in descending order using arsort().

Step 9: Display the file names with their modification date and time.

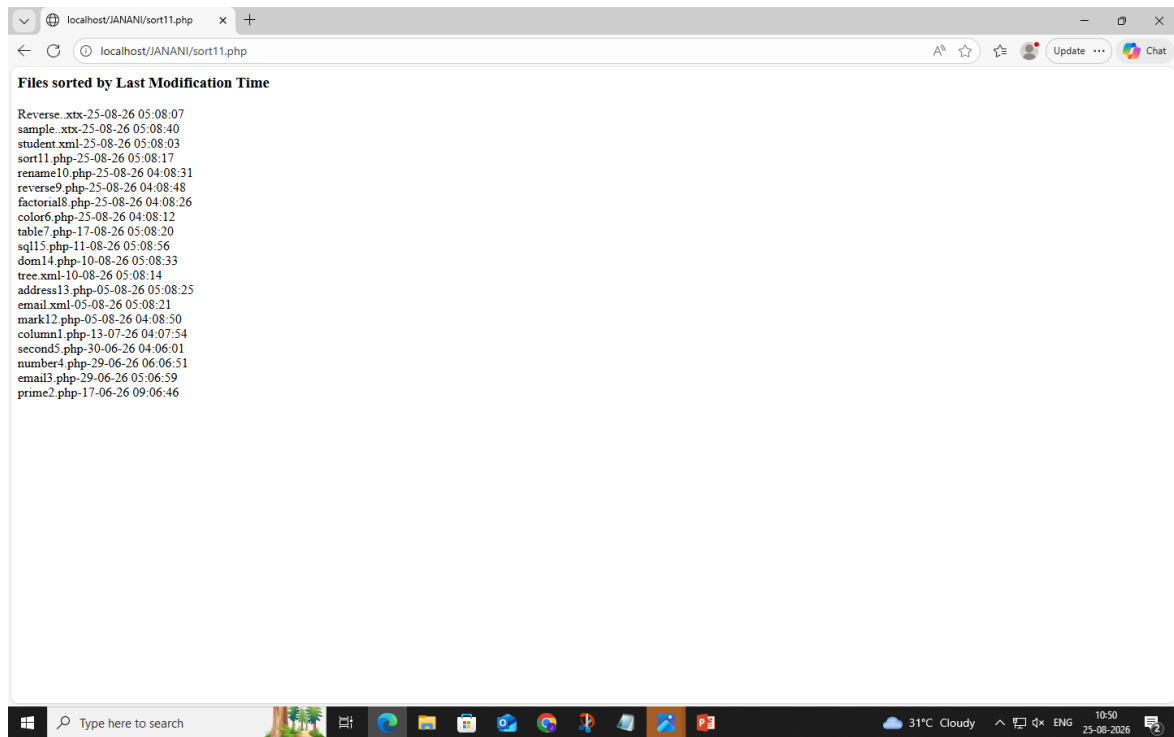
Step 10: Close the directory using closedir().

Step 11: Stop the program.

PROGRAM:

```
<?php
$files=array();
$dir=opendir(".");
while(($file=readdir($dir))!==false)
{
if($file!="."&&$file!="..")
{
$files[$file]=filemtime($file);
}
}
arsort($files);
echo"<h3>files sorted by last modification time</h3>";
foreach($files as $file => $time){
echo $file."-".date("d-m-y H:i:s",$time);
echo"<br>";
}
closedir($dir);
?>
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.

Aim:

To create a student marksheet system that can add, delete, update and view student records using PHP and MySQL.

Algorithm:

Step 1: Start the program.

Step 2: Connect to the MySQL database using `mysqli_connect()`.

Step 3: Check whether the database connection is successful.

Step 4: Insert a student record using the INSERT query.

Step 5: Display a message if the record is inserted successfully.

Step 6: Modify the Maths mark using the UPDATE query.

Step 7: Display a message if the record is updated successfully.

Step 8: Delete the student record using the DELETE query.

Step 9: Display a message if the record is deleted successfully.

Step 10: Close the database connection.

Step 11: Stop the program.

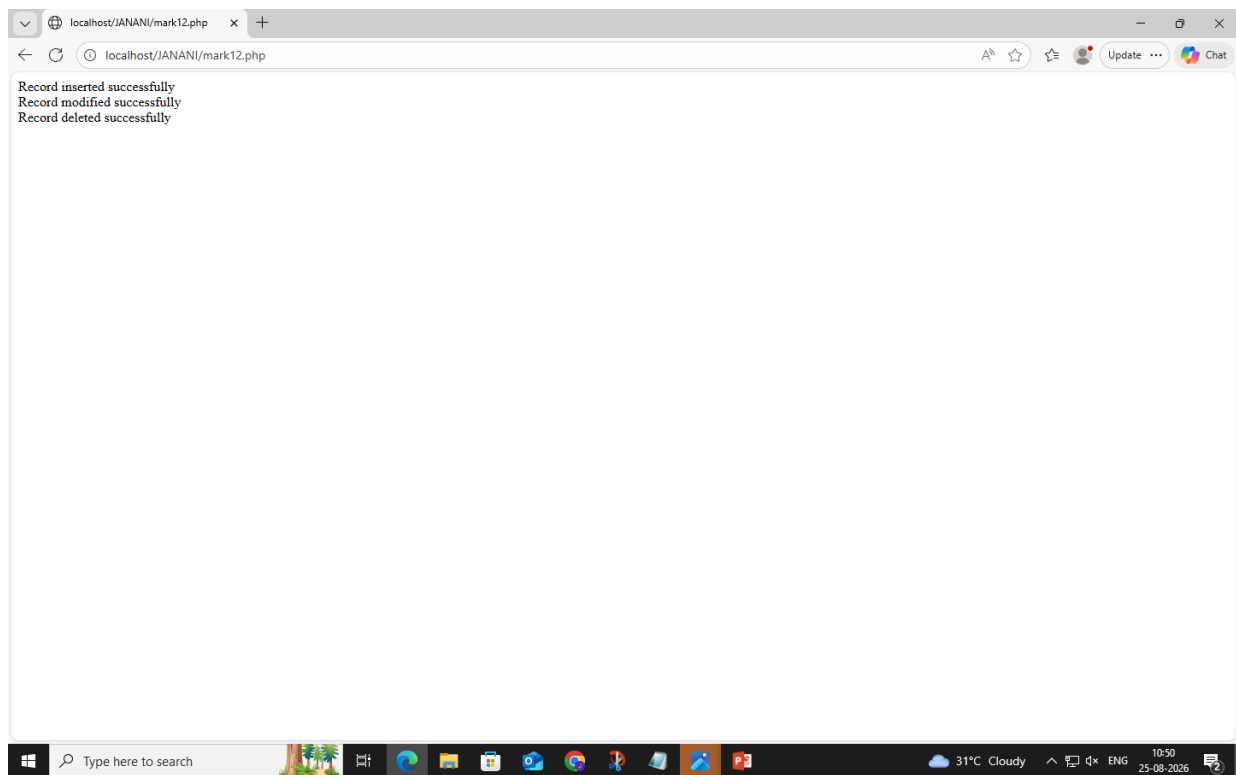
PROGRAM:

```
<?php
$conn=mysqli_connect("localhost","root","","studentdb");
if(!$conn)
{
die("connection failed");
}
$sql="INSERT INTO studentdb VALUES(101,'arun',85,90,95)";
if(mysqli_query($conn,$sql)){
echo"record inserted successfully<br>";
}
$sql="UPDATE studentdb
SET maths=98
WHERE regno=101";
if(mysqli_query($conn,$sql)){
echo"record modified successfully<br>";
}
$sql="DELETE FROM studentdb
WHERE regno=101";
if(mysqli_query($conn,$sql)){
echo"record deleted successfully";
}
mysqli_close($conn)
?>
```

SQL COMMAND

```
CREATE DATABASE studentdb;  
USE studentdb;  
CREATE TABLE student  
(  
    Regno INT PRIMARY KEY,  
    Name VARCHAR(30),  
    Tamil INT,  
    English INT,  
    Maths INT);
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.

Aim:

To write a PHP program to read email addresses from an XML file using SimpleXML.

Algorithm:

Step 1: Start the program.

Step 2: Create an XML file containing names and email addresses.

Step 3: Load the XML file using `simplexml_load_file()`.

Step 4: Access each `<person>` element using a foreach loop.

Step 5: Retrieve the email address from the `<email>` element.

Step 6: Display the email address.

Step 7: Repeat Steps 4 to 6 for all persons.

Step 8: Stop the program.

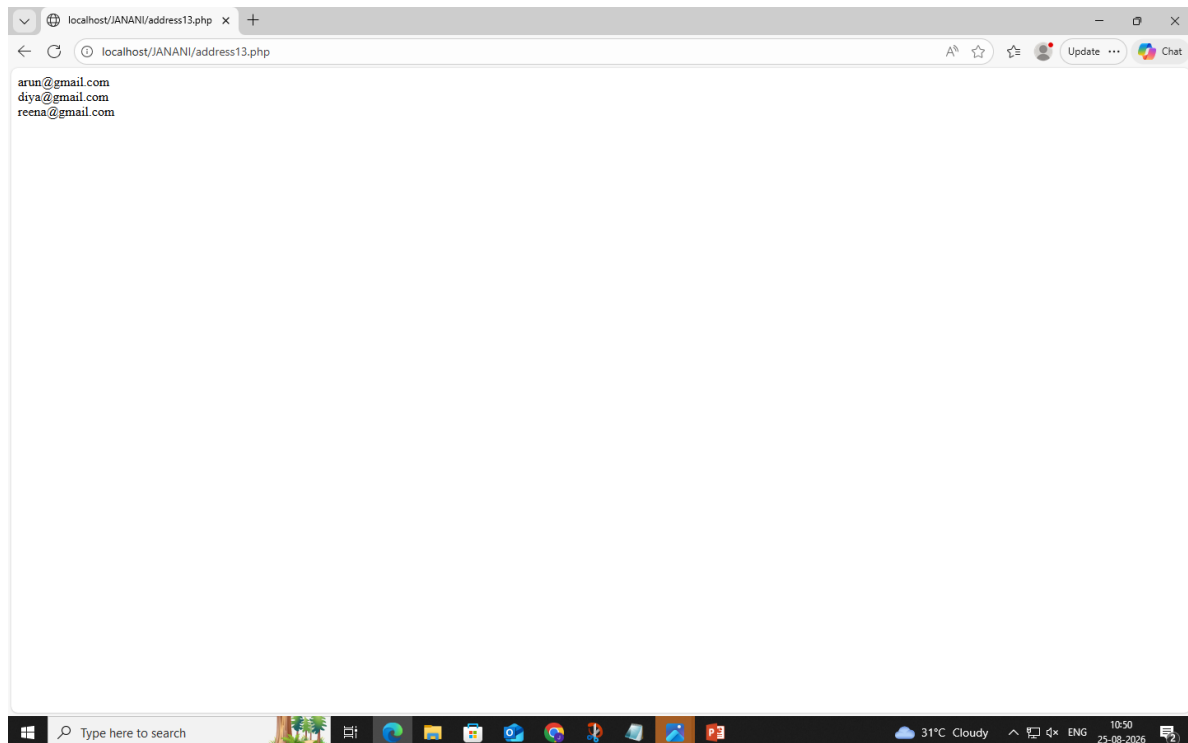
PROGRAM:

```
<?php
$xml=simplexml_load_file("email.xml");
foreach($xml->person as $person){
echo $person->email;
echo"<br>";
}
?>
```

XML FILE

```
<?xml version="1.0"?>
<emails>
<person>
<name>arun</name>
<email>arun@gmail.com</email>
</person>
<person>
<name>diya</name>
<email>diya@gmail.com</email>
</person>
<person>
<name>reena</name>
<email>reena@gmail.com</email>
</person>
</emails>
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.

AIM:

To write a PHP program to read student details from an XML file using DOM methods.

ALGORITHM:

Step 1: Start the program.

Step 2: Create a DOMDocument object.

Step 3: Load the XML file using the load() method.

Step 4: Retrieve the <name> element using getElementByTagName().

Step 5: Display the first student's name.

Step 6: Access the <student> element through the document element.

Step 7: Retrieve the required child node and display its value.

Step 8: Access the child nodes using childNodes.

Step 9: Retrieve and display the required node value.

Step 10: Stop the program.

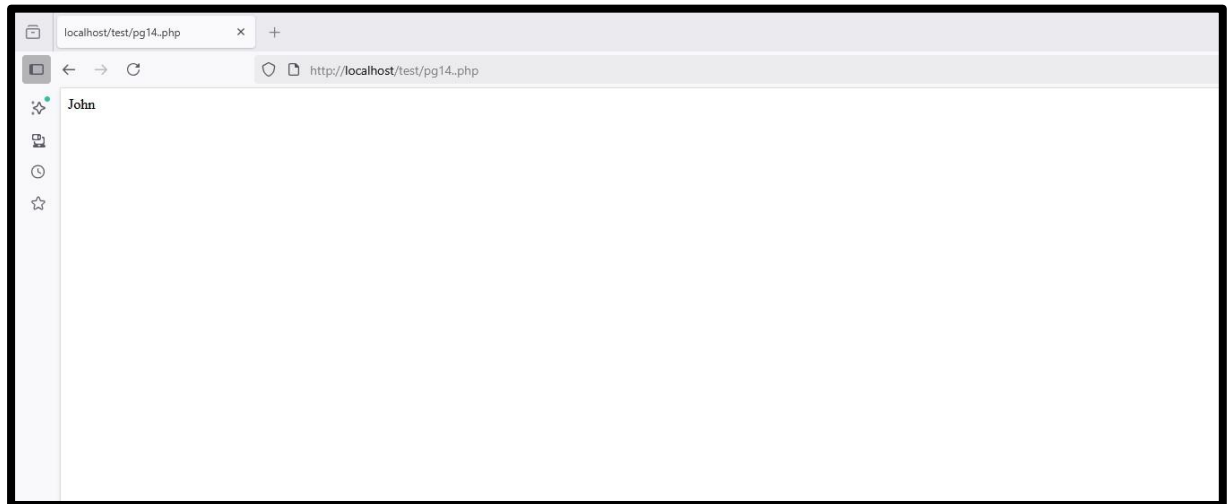
PROGRAM:

```
<?php
$dom=new DOMDocument();
$dom->load("tree.xml");
echo"method1:";
echo $dom->getElementsByTagName("name")->item(0)->nodeValue;
echo"<br>";
echo"method2:";
echo $dom->documentElement->getElementsByTagName("student")->item(0)-
>firstChild->nodeValue;
echo"<br>";
echo"method3:";
echo $dom->getElementsByTagName("student")->item(0)->childNodes->item(1)-
>nodeValue;
echo"<br>";
?>
```

XML CODE:

```
<?xml version="1.0"?>
<students>
  <student>
    <name>John</name>
    <course>BCA </course>
  </student>
  <student>
    <name>Kavin</name>
    <course>BCA </course>
  </student>
</students>
```

OUTPUT



RESULT:

Thus, the PHP program has been executed successfully.

Aim:

To write a PHP program to retrieve student records from MySQL and create an XML file.

Algorithm:

Step 1: Start the program.

Step 2: Connect to the MySQL database using `mysqli_connect()`.

Step 3: Execute the SELECT query to retrieve student records.

Step 4: Create a DOMDocument object.

Step 5: Create the root element `<students>`.

Step 6: Retrieve each database record using `mysqli_fetch_assoc()`.

Step 7: Create a `<student>` element for each record.

Step 8: Create `<regno>`, `<name>` and `<course>` elements.

Step 9: Insert the corresponding database values into the XML elements.

Step 10: Add each `<student>` element to the `<students>` element.

Step 11: Save the XML document as `student.xml`.

Step 12: Display **"XML file created successfully"**.

Step 13: Close the database connection.

Step 14: Stop the program.

PROGRAM:

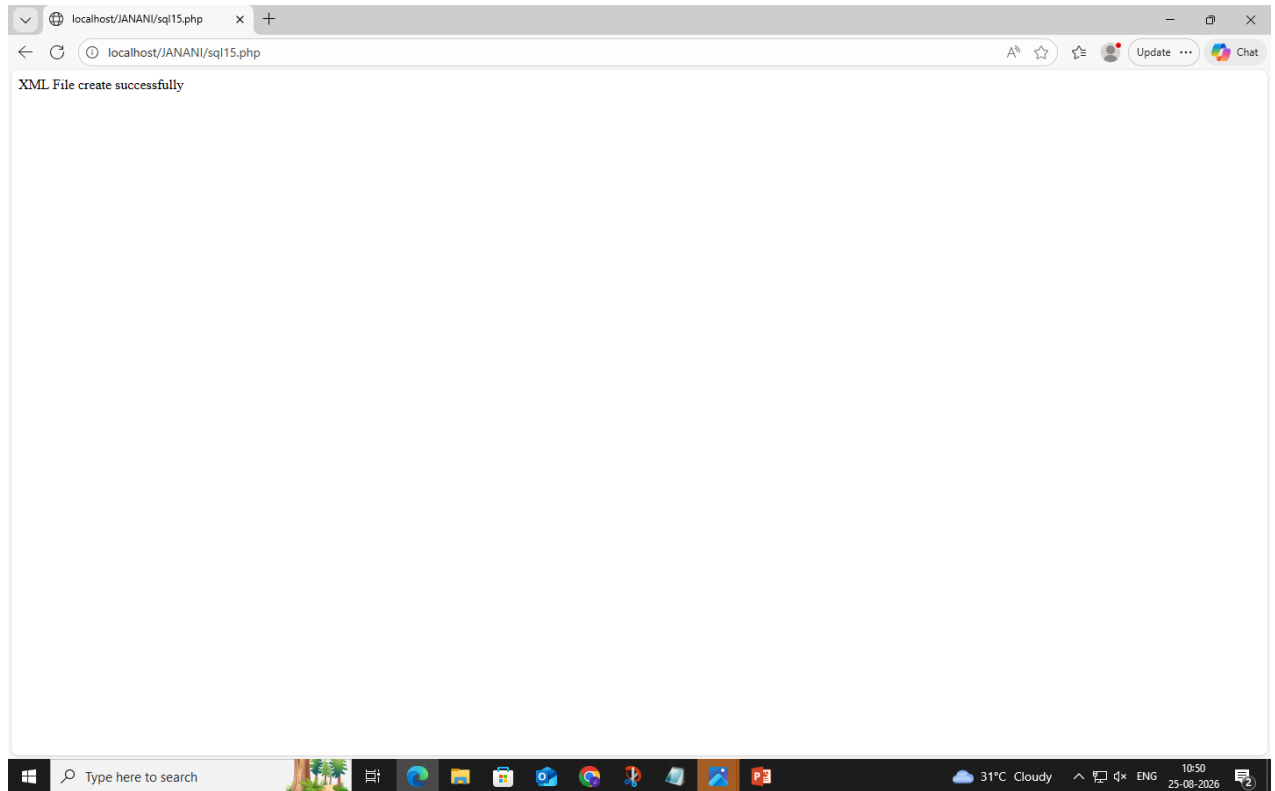
```
<?php
$conn = mysqli_connect("localhost", "root", "", "student21");
$result = mysqli_query($conn, "SELECT * FROM student");
$xml = new DOMDocument("1.0");
$xml->formatOutput = true;
$students = $xml->createElement("students");
$xml->appendChild($students);
while ($row = mysqli_fetch_assoc($result)){
    $student = $xml->createElement("student");
    $student->appendChild($xml->createElement("regno", $row['regno']));
    $student->appendChild($xml->createElement("name", $row['name']));
    $student->appendChild($xml->createElement("course", $row['course']));
    $students->appendChild($student);
}
$xml->save("student.xml");
echo "XML file created successfully";
mysqli_close($conn);
?>
```

Generated XML File:

student.xml :

```
<?xml version="1.0"?>
<?xml version="1.0"?>
<students>
  <student>
    <regno>101</regno>
    <name>Arun</name>
    <course>BCA</course>
  </student>
  <student>
    <regno>102</regno>
    <name>Divya</name>
    <course>BCA</course>
  </student>
  <student>
    <regno>103</regno>
    <name>Kavin</name>
    <course>BCA</course>
  </student>
</students>
```

OUTPUT:



RESULT:

Thus, the PHP program has been executed successfully.